RIT ALGO2e: Strategy Report

Main strategy: `algo2e_fin.py`

[Team names]

[Course] | Simulation of 2 October 2026

We traded the ALGO2e case with one algorithm, `algo2e_fin.py`, which combines a market maker, a trend-following engine and a risk layer. We also ran two variants of the same code on separate accounts to isolate where performance came from: `dir.py` (trend engine only) and a pure market-making build (trend engine switched off). This report answers the four questions in turn. All numbers come from the logs our bot wrote during the run; the section on our Python script explains the logging.

(a) What was your strategy going in?

The question we started from. ALGO2e is a liquidity-provision case: three stocks (CNR around $160, RY around $100, AC around $25), a 25,000-share gross and net limit with fines for breaches, and a fee schedule that rewards resting orders on two of the three names. The obvious way to make money is market making: quote both sides, earn the bid–ask spread and the maker rebate, and keep inventory small. The obvious way to *lose* money is the same thing in a trending market, because a market maker keeps buying a falling price. Nobody told us which kind of path the prices would follow, so the strategy had to be right in both worlds without us switching anything by hand during the case.

Our working assumption and our hedge. Our working guess was that prices would mostly mean-revert around an anchor, with occasional trends or shocks that the instructor could inject. We therefore designed a *regime-switching* system rather than a single strategy:

1. **Market maker (base income in calm markets).** Quotes on every stock, sized by volatility state, with an inventory brake that pulls the position back to zero.
2. **Trend engine (insurance against the assumption being wrong).** When a stock moves steadily in one direction, the market maker stands aside and the trend engine takes a large position in the direction of the move, held with a volatility-scaled trailing stop.
3. **Risk layer (non-negotiable).** Every order is checked against worst-case exposure before it is sent, and the bot only ever uses limit orders. Our objective was profit *with zero fines*.

Where we expected the money to come from. In order of expectation: spread and rebate capture from market making; trend profits if the market broke out; and a small amount from a "sweep catcher" (resting orders 3%, 8% and 15% from fair value that fill only when someone sweeps a thin book). The economics of the first source are thin: with a one-tick spread, a 1,000-share round trip earns about $10 plus a few dollars of rebate, so it only pays at high volume and with very few losing trades. That observation is why we did not want to rely on it alone.

How the bot decides which regime it is in. Each stock carries its own measurements, updated on every pass of the loop (up to 40 times a second):

- **EMA gap**: fast (1 s) minus slow (8 s) exponential moving average of the mid-price, in ticks. A persistent gap means the price has run away from its recent past.
- **Kaufman efficiency ratio** over 10 s: |net move|/path length. It is near 1 for a straight-line move and near 0 for chop.
- **Bollinger band width** (30 s window) relative to its own 2-minute baseline, giving a calm / normal / stressed volatility state.
- **Volatility** (mean absolute 0.5 s move) and a **t-statistic of drift** over 20 case ticks for slow trends.

A stock is labelled TREND when the efficiency ratio is at least 0.45 and the move is at least 12 ticks, and it stays TREND until the ratio falls below 0.33 (hysteresis, so the label does not flicker). The label alone never trades; the trend engine also needs the EMA gap above $\max(12 \text{ ticks}, 5\sigma)$, a spread of at most 6 ticks, and the signal to persist for 0.8 s. Thresholds are in units of volatility so the same rules work on a \$25 and a \$160 stock.

What we deliberately did not do. We built a directional mean-reversion tilt (hold inventory long near the bottom of the Bollinger band and short near the top) and tested it at 8,000 and 12,000 shares. It lost to plain inventory-to-zero market making in our tests, because it holds inventory exactly when a breakout starts. It was therefore switched off for the graded run (`MR_MAX_INV = 0`). Section (d) estimates what it would have added.

How we arrived at this design. The rules came from our own practice sessions and from an offline test harness built into the script (described with the script in Section (c)). Practice sessions showed, for example, that market orders into a thin AC book could lose tens of thousands of dollars in a second, which is why every aggressive order is a limit order capped within 2% of fair value; and that a bot reading its own quotes as "the market" walks its own prices away ("pennying itself"), which is why our own orders are stripped from the book before any price is computed.

(b) What changes did you make to handle multiple securities and fees/rebates?

Nothing about the securities is hard-coded. At start-up the bot reads `/securities` and `/limits` from the RIT API. From `/securities` it takes the list of tradeable tickers, each ticker's tick size (`quoted_decimals`), its taker fee and its maker rebate. From `/limits` it takes the gross and net limits and the fine schedule, and derives its own caps from them. The same script therefore runs unchanged on any number of securities or a different limit.

One state machine per security, one shared risk budget. Each stock has its own signals, regime label, mode (market making, trend, or "cut" while exiting a trend), stops, loss pauses and learned entry threshold. What the stocks share is the risk budget:

- **Aggregate worst-case exposure.** Before any order, the bot assumes every resting order, every in-flight order and the new order fill at once, summed across all stocks, and requires gross and net to stay under 93% of the limit (23,250 shares).

- **Capacity allocation.** The trend engine's size on one stock is $0.95 \times$ cap minus the positions held in the other stocks, so a trend in one name cannot crowd the limit already used by another.
- **Priority under contention.** When room is tight, aggressive orders go first, then the quotes of the stock with the healthiest wide spread (the most profitable to quote), then the remaining quotes, then the sweep ladder.
- **Per-stock caps.** Market-making inventory is capped at 9,000 shares per stock so that no single name absorbs the whole budget.

Books for all stocks are fetched in parallel (a 16-thread pool) so that adding a security does not slow the loop for the others.

Fees and rebates: what the schedule implies. Table 1 restates the schedule as a fraction of the one-tick ($0.01) spread, which is the number that matters for a market maker.

Stock	Maker rebate (per share)	Taker fee (per share)	Rebate as % of 1 tick	Taker fee as % of 1 tick
CNR	+$0.0023	$0.0027	+23%	27%
AC	+$0.0011	$0.0015	+11%	15%
RY	−$0.0020 (pays)	−$0.0014 (earns)	−20%	−14%

Table 1: Fee schedule from the case brief; the bot also reads it from the server at start-up. On CNR the maker rebate alone is almost a quarter of the spread; on RY it is reversed.

Three design choices follow from this table:

1. **Passive-first execution.** Market making uses only resting orders, which earn the rebate on CNR and AC. We cross the spread only when we must (stops, trend entries and exits, end-of-case unwind), always with capped limit orders.
2. **Size where the economics are best.** Full size (5,000 shares a side) is quoted only in calm markets or in a "healthy wide" spread of 2–6 ticks, where spread plus rebate pays for the inventory risk. On a two-tick spread we step inside only on the side that reduces inventory.
3. **RY is structurally different.** Resting on RY costs money, so RY market making is weaker by construction, while crossing on RY earns a small rebate. Our trend trades in RY were executed with aggressive limit orders, which is the cheaper side on that stock.

To be precise about the scope: in this version the fee schedule shaped the design rather than entering the quoting formula. Making the quote edge explicitly fee-aware (for example, not quoting RY passively at all, or requiring expected edge to exceed the fee) is the first change we would make.

The exchange is asynchronous, and the bot is built for it. RIT confirms orders and updates positions with a delay of a fraction of a second. The bot therefore keeps its own registry: a just-sent order counts as live for 1.5 s even if RIT has not listed it yet, a vanished order still counts towards exposure for 1.0 s, and leftovers of aggressive orders are cancelled after 0.6 s. It also measures latency live and moves quotes away from a price that is running faster than an order can arrive.

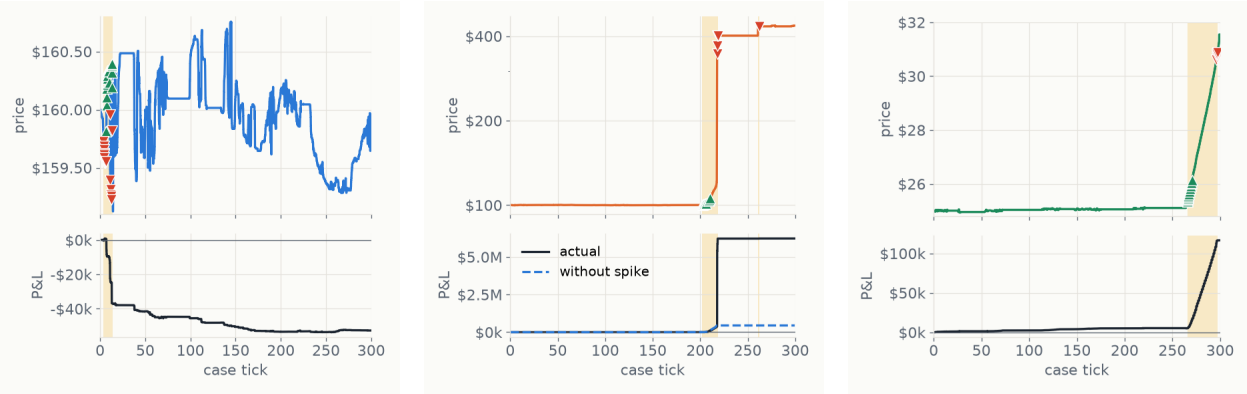
(c) What happened during the simulation? What was your size strategy?

The market we actually got. For the first 200 case ticks all three prices were strongly mean-reverting, which supported our working assumption (Table 2). A random walk has a variance ratio

of 1 and zero return autocorrelation; all three stocks were far below that. After tick 200 the picture changed: RY broke out and AC trended, while CNR stayed in its band for the whole case (Figure 1).

Ticks 0–199	Mean	Std. dev.	Range	Lag-1 autocorr. of returns	Variance ratio (10 ticks)
CNR	\$160.06	\$0.29	\$159.19–160.75	−0.21	0.25
RY	\$100.01	\$0.08	\$99.88–100.21	−0.33	0.19
AC	\$25.03	\$0.04	\$24.95–25.09	−0.28	0.26

Table 2: Before tick 200 all three prices behaved like mean-reverting series, not random walks.



(a) CNR: −\$53k **(b)** RY: +\$6.22M **(c)** AC: +\$117k

Figure 1: Price path (top) and cumulative P&L (bottom) for each stock. Green and red triangles are trend-engine buys and sells; the shaded band is trend mode. In the RY panel the dashed line is the P&L without the tick-217 spike.

Timeline.

- **Ticks 3–13, CNR.** The trend engine took three positions (short, long, short) that reversed immediately, for −\$36.7k. In the first seconds of the case CNR's book was thin (spreads of 11 to 100 ticks) and prices jumped 30–100 cents per tick, while our volatility estimate was still at its starting value. The efficiency ratio read 1.0 on a handful of observations. The engine mistook start-up noise for a trend. Its adaptive threshold then did its job: each loss raised CNR's entry bar by 1.4×, three losses in a row locked the engine out of CNR, and it never fired on CNR again.
- **Tick 38.** The account-level circuit breaker fired once (P&L \$40k below peak): flatten and pause for 8 s.
- **Ticks 0–199.** Quiet, mean-reverting markets. Market making traded actively but did not make money (explained below).
- **Ticks 200–216, RY.** RY started drifting up from \$100. The efficiency ratio jumped from 0 to 1.0 at tick 200, the label became TREND, and at tick 201.6 the EMA gap passed 12 ticks for 0.8 s and the trend engine took over (Figure 2). It built 22,062 shares between \$100.2 and \$105.5 by tick 210 and held them while the price drifted to \$118.
- **Tick 217, RY.** The RY book became one-sided: the asks disappeared and the bids jumped to \$346 and then \$402 (we understand this followed Group 12's activity). The engine exited

in five orders over about four seconds, selling all 22,062 shares into those bids at an average of roughly $383.

- **Ticks 265–296, AC.** AC began a straight climb from $25.1 to $31.5. The engine entered at tick 265, held about 22,000 shares and closed in the end-of-case unwind.

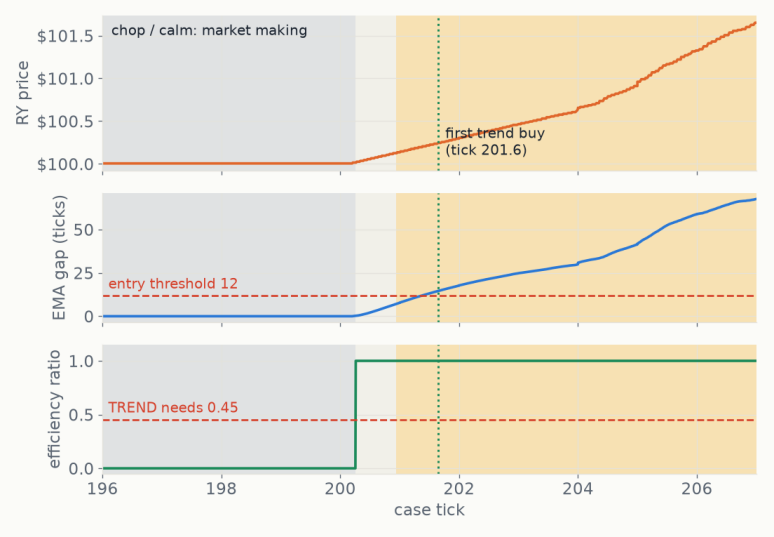


Figure 2: Regime detection on RY around the breakout. The efficiency ratio jumps to 1.0 at tick 200; the trend engine buys once the EMA gap has stayed above 12 ticks for 0.8 s.

Why market making did not pay in a mean-reverting market. This is the most instructive result of the run. Mean reversion in the price is not the same thing as profitable market making. The spread was one tick on all three stocks, so a successful round trip earned about $0.01 per share plus rebate, while the inventory stop cuts at $\max(5 \text{ ticks}, 3\sigma)$. With a payoff of roughly $+1$ tick on a win and -5 ticks on a stop, the market maker needs about five wins in every six trades just to break even, and prices here moved in jumps of several cents rather than one-cent wiggles. CNR alone recorded 86 inventory stops. Adverse selection makes this worse: the resting orders that fill are disproportionately the ones a moving price runs through. The outcome was a small, steady bleed (Section (d)), confirmed independently by the market-making-only build.

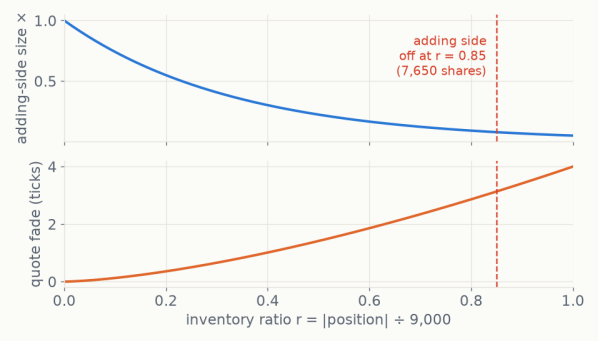
Size strategy. Our sizing rule was simple: size up only when the evidence is strong or the risk is small, and size down as risk builds.

Component	Rule
Market-making quotes	5,000 shares a side when calm or in a healthy 2–6 tick spread; 3,000 normally; half that when stressed; 2,000 in thin books.
Inventory brake	With $r = \lvert\text{position}\rvert/9{,}000$: adding-side size $\times e^{-3r}$; adding-side quote moves away by $\text{round}(4r^{1.5})$ ticks; adding side switched off at $r \geq 0.85$ (7,650 shares); reducing side $\times(1+r)$, capped at 5,000.
Trend engine	First order 60% of the available cap; grows with signal strength up to 95% of the cap shared across stocks (about 22,000 shares); orders of at most 5,000.
Sweep ladder	1,500 / 1,500 / 2,000 shares at 3% / 8% / 15% from fair value, only while the position on that side is below half the per-stock cap.
Session governor	Quote size halved if P&L is below −$25k and quartered below −$50k.

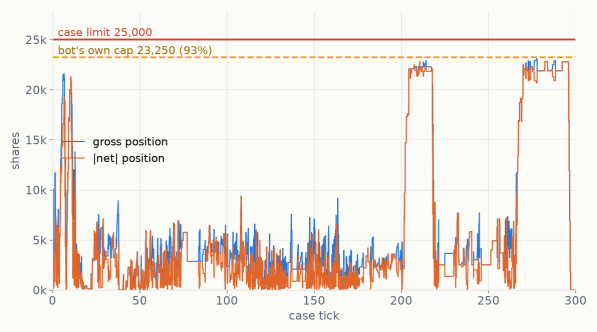
Table 3: Sizing rules in `algo2e_fin.py`.

The inventory brake is a discrete version of the inventory-skew idea in the Avellaneda–Stoikov market-making model: as inventory grows, the bot quotes less on the side that would add to it and moves that quote away, while making the reducing side more attractive (Figure 3a).

Limits and unwinding. The limit plan had four layers: the worst-case pre-trade check at 93% (23,250 shares); a safety net that cuts the largest position if the actual position ever passes 96% (24,000); the $40k drawdown circuit breaker; and an end-of-case schedule that stops adding risk about 20 s before the close and crosses to flat about 8 s before it (both scaled to case length). After any trend exit the stock goes into a "cut" mode that sells the position down with capped limit orders. Over the whole run the peak gross position was 23,088 and the peak net position 22,837, with no logged moment above 24,000 (Figure 3b).



(a) Inventory brake



(b) Gross and net position against the limit

Figure 3: Left: how quote size and quote distance respond to inventory. Right: the position never reached the 25,000 limit or the bot's own 23,250 cap.

Our Python script and how to use it

Files. `algo2e_fin.py` is the main strategy (submitted with this report). `dir.py` is the same code with market making disabled and earlier trend entries; the market-making-only build (`mm.py`) is the same code with the trend engine disabled, equivalent to running `algo2e_fin.py -no-dir`. The script needs Python 3 and the `requests` package; everything else is standard library.

Running it against RIT. Start the RIT client, enable the API, note the API key and port, then run:

```
python algo2e_fin.py <API_KEY> <PORT> <CASE_MINUTES>
python algo2e_fin.py abcde 15000 20 # example: 20-minute case
python algo2e_fin.py --key abcde --port 15000 # case length learned from the clock
```

The case length is optional; without it the bot learns seconds-per-tick from the case clock and scales its time windows accordingly. Useful options: `-no-dir` (market making only), `-size N` (quote size per side), `-no-flatten-on-exit`. Ctrl-C cancels every order and flattens the book before exiting. Every configuration constant sits in one block at the top of the file and can also be overridden from the environment for experiments, for example `CFG_MR_MAX_INV=8000` switches on the mean-reversion tilt.

What it prints and logs. The console shows each fill, mode change and stop, and once a second a status line per stock with its position, quotes, mode, regime, volatility state, efficiency ratio, t-statistic and Bollinger z-score, plus measured latency and the API's rate-limit count. Two CSV files are written to `./logs/`: `v3_ticks_<time>.csv` (one row per loop: prices, signals, positions, modes and P&L for every stock) and `v3_events_<time>.csv` (every order, cancel, fill and decision with its reason). All the analysis in this report was produced from those two files.

Offline testing. The script contains a fake exchange that imitates RIT's behaviour, including order latency (0.25 s), position-report lag (0.30 s), maker rebates and taker fees, and eleven market scenarios. It runs without RIT:

```
python algo2e_fin.py --mock spike --seconds 90
# scenarios: chop trend spike reversal whipsaw meanrev arch crash flood grind sweep
```

At the end it prints the true P&L and the maximum gross and net position. We used it to check that every scenario respects the limits and that no scenario produces an outsized loss. Re-running the final script through all eleven scenarios gives Table 4.

Scenario	What the fake market does	P&L	Max gross	Max net
chop	Random small moves around a fixed level	−$932	11,500	11,500
meanrev	Prices pulled back to their starting level	+$440	14,500	14,500
grind	CNR slides slowly by $1.30	−$247	14,093	13,493
trend	AC rises $0.25/s for a third of the run; CNR falls later	+$124,483	23,200	22,987
arch	AC and CNR rise and then fall (inverted U)	+$91,151	23,162	23,162
reversal	AC rises then reverses; RY has fake-out jolts	+$85,579	23,187	22,587
crash	RY sells off over several minutes	+$41,393	22,987	22,887
sweep	Market orders sweep a thin AC book	+$49,071	14,700	8,200
spike	Repeated spikes in the AC book	+$29,281	13,300	12,700
flood	Heavy one-sided flow hits our quotes	+$12,190	17,400	15,700
whipsaw	AC and CNR alternate 4 s up, 4 s down	−$44,328	20,600	20,600

Table 4: Final script on the built-in fake exchange, 90 s per scenario (seed 1). Every run ended flat and below the 23,250 cap. Results vary slightly between runs because the loop is timing-dependent.

The offline results have the same shape as the live run. Calm and mean-reverting markets are roughly break-even for the market maker, the strategy earns when a stock trends, and the worst case is a whipsaw: short trends that reverse before a trailing stop can pay. That is exactly the

failure we saw on CNR in the first seconds of the live case, and it is why a warm-up filter on the trend engine is the first improvement on our list.

(d) What was your P&L?

Headline. The main strategy finished at **$6,288,771**, measured as the server's net liquidation value. It equals the sum of the three stocks' P&L to the cent, which is what we would expect if no fines were charged. The two variants ran on separate accounts in the same market and are reported separately (Table 5).

	CNR	RY	AC	Total
`algo2e_fin.py` (main: market making + trend)	−$52,646	$6,224,827	$116,590	**$6,288,771**
`dir.py` (trend engine only)	−$77,895	$6,238,173	$118,130	$6,278,408
Market making only	−$28,719	−$6,147	$6,434	−$28,433

Table 5: Final P&L by stock for the three builds (server values).

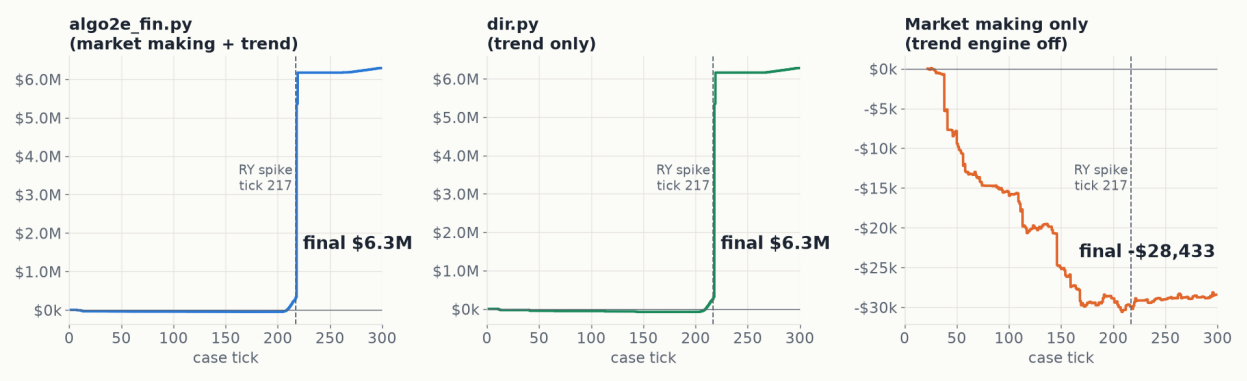


Figure 4: Total P&L through the case for the three builds. The dashed line marks the RY event at tick 217.

Where it came from. We split the main strategy's P&L using the mode the bot was in when each position was held.

Component	P&L
RY: selling the trend position into the tick-217 dislocation	+$5,777,656
RY: trend ride from $100 to the last normal price	+$450,870
AC: trend ride and exit	+$111,511
AC / RY / CNR: market making	+$5,079 / −$3,698 / −$16,152
CNR: early trend calls (ticks 3–13)	−$36,494
Total	$6,288,771

Table 6: Decomposition of the main strategy's P&L.

The trend engine was in control for under 7% of the time on any stock (about 2% on CNR, 4% on RY, 6% on AC) and produced all of the profit, net +$6.30M; market making was active the rest of the time and finished at about −$15k. The comparison across builds tells the same story under controlled conditions: the same trend engine produced almost the same result with or without

market making, while market making on its own lost money. `dir.py`'s faster entries (8 instead of 12 ticks) cost it more on CNR, a direct illustration of the trade-off between catching moves early and reacting to noise.

With and without the RY event. About 92% of the profit came from one event. To show what the strategy earned on its own terms, we re-valued the RY exit at the last bid before the book broke ($121.08), keeping everything else unchanged. The main strategy would then have finished at about **$511k**, and between $244k and $559k for exit prices from $109 to $123 (Figure 5). This estimate still includes the RY drift that preceded the event, so it is the optimistic reading. Ticks 0–199 alone were a loss of about $52k.

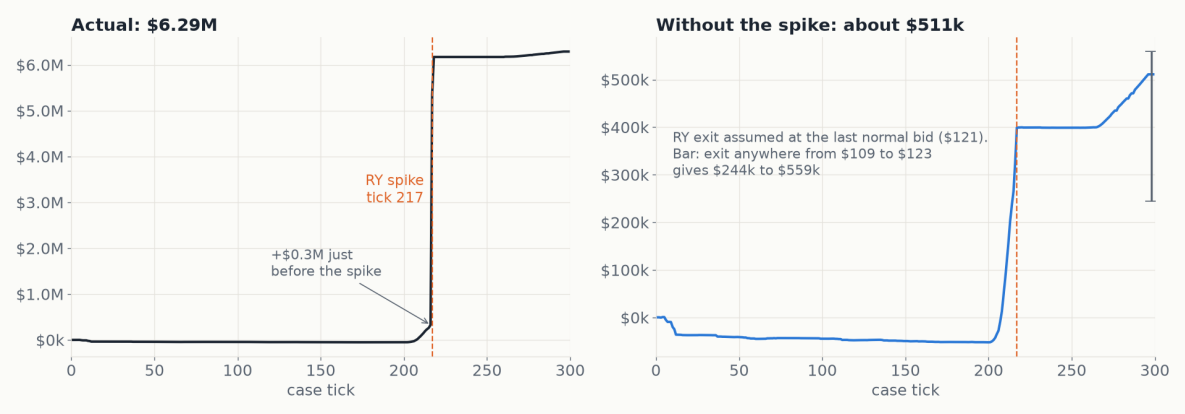


Figure 5: Actual P&L (left) and the same run with the RY exit valued at the last normal bid (right).

What the mean-reversion tilt would have added. Replaying the logged Bollinger z-scores and regime labels, and holding the position the tilt would have targeted, gives +$37k at 8,000 shares and +$56k at 12,000 shares before costs, and roughly +$5k to +$20k at 8,000 shares after half to one tick of cost on about three million shares traded. Almost all of it falls in ticks 0–199, because the tilt switches itself off when a stock is in TREND, so it would not have held a short into the RY move. It is an upper bound (it assumes the target position is reached instantly) and below 1% of the total, which is consistent with our decision to leave it off.

How we read the result. The P&L is dominated by one event, and we do not claim we predicted it. What the design did was put the right position on by rule before the event: regime detection moved RY into trend mode within about one case tick of the drift starting, the trend engine sized up quickly to 88% of the limit, and the wide, volatility-scaled trailing stop kept the position on until the book broke. Equally, the run exposed the two weaknesses we would fix first: the trend engine needs a warm-up period before it is allowed to trade (the early CNR losses), and market making should be fee-aware and limited to stocks where resting earns a rebate. Throughout, the position stayed below the limit and no fines were charged.